

QFOR — 이차곡면 fit 기반 Outlier 제거

Quadric-Fitting Outlier Removal: 곡면 위에서 "특 튀어나온" 점만 정확히 골라내기

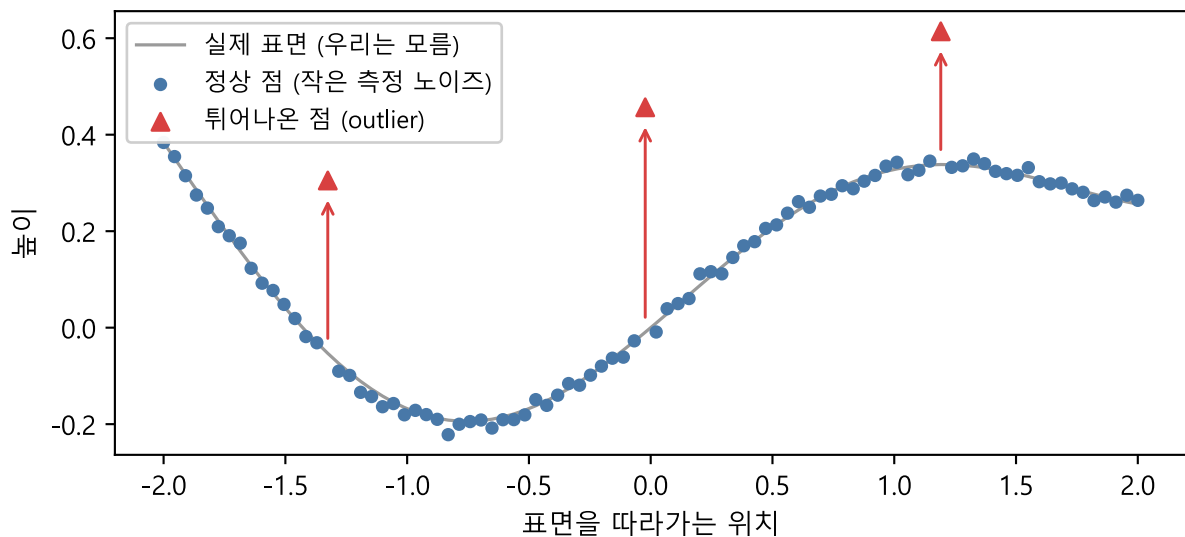
Orange 엔진 `orange::modes` · Geometry ▶ Filter ▶ "Outlier: QFOR" · 초보자용 단계별 설명자료 · 2026-07

1. 어떤 문제를 푸는가

3D 스캐너로 물체를 찍으면 결과는 **point cloud**(점 구름) — 수십만 개의 (x, y, z) 점입니다. 스캐너는 완벽하지 않아서 두 종류의 오차가 섞입니다.

- **측정 노이즈** — 모든 점이 실제 표면에서 아주 조금씩(μm ~수십 μm) 떨어져 있음. 정상이며, 제거 대상이 아닙니다.
- **Outlier(이상점)** — 반사, 먼지, 다중 반사 경로 때문에 표면에서 **확연히 튀어나온** 점. 이후의 메쉬 재구성·측정·정합을 모두 망가뜨리므로 제거해야 합니다.

문제: 표면에서 '특 튀어나온' 점을 어떻게 찾을까?



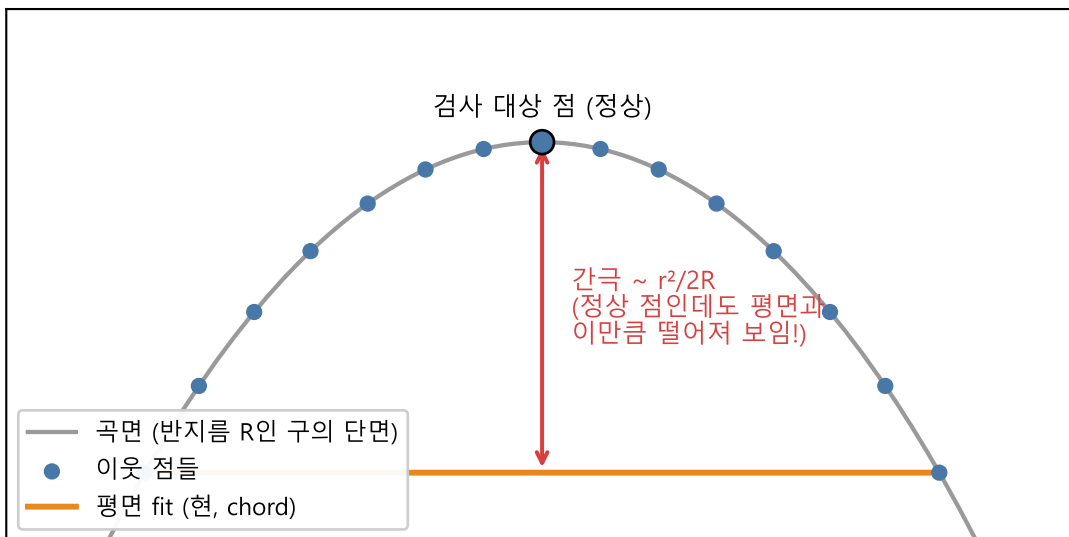
어려운 이유는 하나입니다: **우리는 실제 표면이 어디인지 모릅니다**. 가진 것은 점들뿐이므로, 각 점이 "튀어나왔는지"를 판정하려면 먼저 그 점 주변의 점들로부터 **기준 표면을 추정**해야 합니다. 기준 표면을 무엇으로 추정하느냐가 알고리즘들의 갈림길입니다.

2. 기존 방법들과 각각의 한계

방법	판정 기준	한계
SOR (Statistical)	이웃 k개까지의 평균거리가 전체 평균보다 비정상적으로 크면 제거	허공에 뜬 고립점 엔 강하지만, 표면 가까이 붙어 튀어나온 점은 이웃 거리가 정상이라 못 잡음
ROR (Radius)	반경 r 안의 이웃 수가 N개 미만이면 제거	같은 이유 — 돌출점도 주변 밀도는 정상인 경우가 많음
PFOR (Plane-Fitting)	이웃들로 평면 을 fit하고, 그 평면에서 먼 점을 제거	표면이 곡면 이면 정상 점도 평면에서 멀어짐 → 곡면을 노이즈로 오검출 (아래 그림)

표면 돌출 검출에는 PFOR의 접근("기준면과의 거리")이 좋습니다. 문제는 기준면을 **평면**으로 잡는 데 있습니다.

PFOR의 함정: 매끈한 곡면 위의 정상 점도 '평면'에서는 멀다



PFOR의 근본 한계

반지름 R인 구 표면에서 이웃 반경이 r일 때, **완벽하게 정상인 점조차** 이웃들의 fit 평면에서 약 $r^2/2R$ 만큼 떨어집니다. 이 값이 진짜 노이즈보다 커지는 순간(곡률이 크거나 이웃 반경이 넓을 때) 평면 기준으로는 정상과 outlier를 구분할 수 없게 됩니다. 문턱값을 높이면 outlier를 놓치고, 낮추면 곡면 전체가 빨강게 물듭니다.

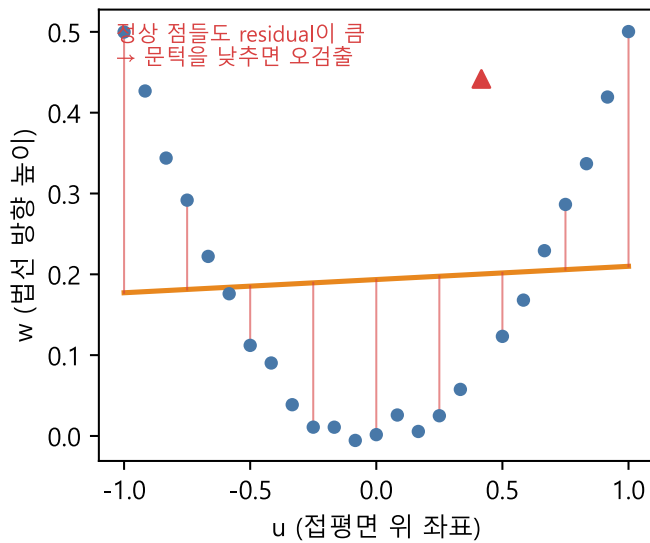
3. 핵심 아이디어: 기준면을 "휘어질 수 있게" 하자

해결책은 단순합니다. 기준면을 1차식(평면)이 아니라 **2차식(이차곡면, quadric)**으로 fit합니다.

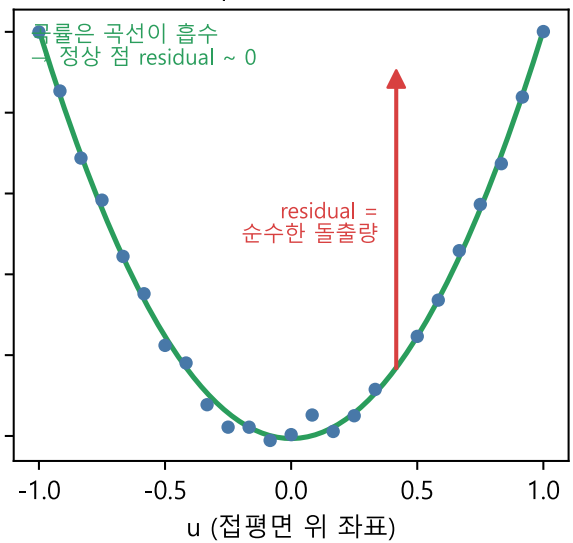
$$w = q(u, v) = a \cdot u^2 + b \cdot uv + c \cdot v^2 + d \cdot u + e \cdot v + f$$

여기서 (u, v) 는 점 주변의 접평면 위 좌표, w 는 표면에서 수직으로 잴 높이입니다(4단계에서 자세히). 2차식은 볼록함·오목함·안장 모양까지 표현할 수 있어서, 매끈한 곡률은 **곡면 계수(a, b, c)가 흡수**합니다. 그러면 fit 후 남는 잔차(residual)는 곡률이 아닌 **순수한 돌출량**이 됩니다.

① 1차 (평면) fit — PFOR



② 2차 (quadratic) fit — QFOR

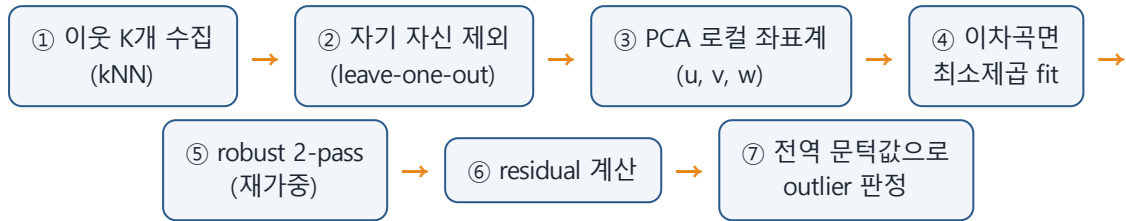


한 줄 요약

PFOR: "평면에서 멀다" = 곡률 + 돌출이 뒤섞인 신호. QFOR: "이차곡면에서 멀다" = 돌출만 남은 깨끗한 신호. 평평한 표면에서는 둘이 같은 답을 내므로 손해가 없습니다.

4. 알고리즘 단계별 해부

QFOR는 모든 점에 대해 독립적으로(병렬) 아래 절차를 수행합니다. 전체 흐름:

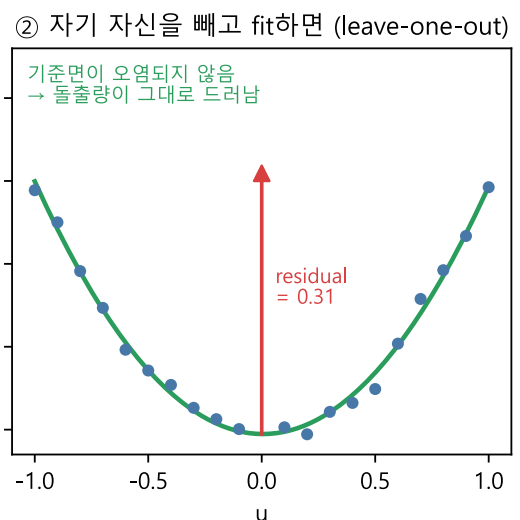
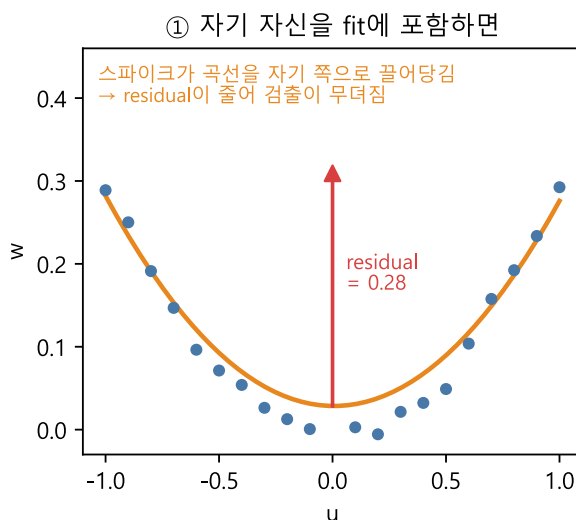


1 이웃 K개 수집 (kNN)

검사할 점 p_i 에서 가장 가까운 K개(기본 24개)의 점을 찾습니다. 점이 수십만 개라도 빠르게 찾을 수 있도록 Orange는 해시 기반 **SparseGrid**(복셀 격자)에 점을 넣어두고 근처 셀만 뒤집습니다. 이 K개가 "주변 표면이 어떻게 생겼는지"를 알려줄 증인들입니다. 이웃이 10개 미만이면(구름 가장자리 등) 미지수 6개를 받을 만하게 못 구하므로 그 점은 판정을 건너뜁니다.

2 자기 자신을 빼기 — leave-one-out

결정적인 디테일입니다. kNN 결과의 첫 번째는 항상 자기 자신인데, **검사 대상을 fit에 포함하면 스파이크가 자기 기준면을 자기 쪽으로 끌어당깁니다.** 판사와 피고인이 같은 사람인 셈이죠. 그래서 기준면(좌표계와 이차곡면 모두)은 **이웃들만으로** 만들고, 검사 대상 점은 완성된 기준면에 대조만 합니다.

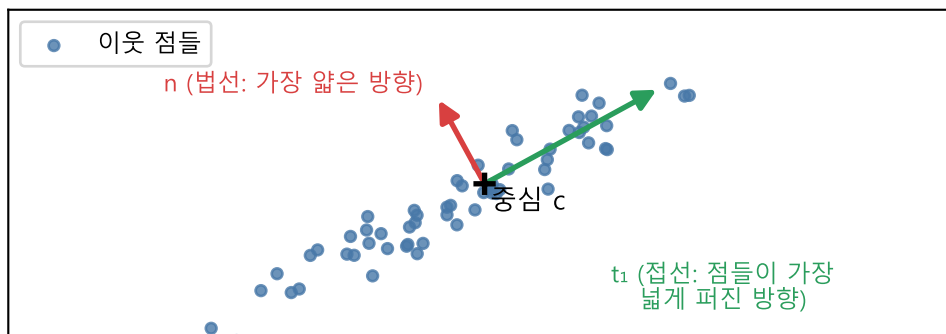


3 PCA로 로컬 좌표계 만들기

"표면에서 수직으로 켜 높이"를 계산하려면 그 자리의 **접선 방향**과 **법선 방향**부터 알아야 합니다. 이웃 점들의 공분산 행렬을 고유분해(PCA)하면: 표면 점들은 접평면 방향으로 넓게, 법선 방향으로 얇게 퍼져 있으므로 —

- 고유값이 **큰** 두 방향 t_1, t_2 = 접평면 (점들이 퍼진 방향)
- 고유값이 **가장 작은** 방향 n = 법선 (표면에 수직)

PCA: 이웃 점들의 퍼진 모양에서 로컬 좌표계를 얻는다



이웃 중심 c 를 원점으로, 각 이웃을 이 좌표계로 변환합니다: $u = (p-c) \cdot t_1$, $v = (p-c) \cdot t_2$, $w = (p-c) \cdot n$. u, v 는 이웃들의 평균 퍼짐 폭으로 나눠 정규화하는데, 구름이 mm 단위든 km 단위든 연립방정식이 수치적으로 안정(well-conditioned)하도록 하기 위해서입니다.

4 이차곡면 최소제곱(least squares) fit

이제 이웃들 (u_j, v_j, w_j) 에 가장 잘 맞는 계수 6개 (a, b, c, d, e, f) 를 찾습니다. "잘 맞는다" = 각 이웃의 예측 오차 제곱합이 최소:

$$\min_{a..f} \sum_j [w_j - q(u_j, v_j)]^2$$

각 이웃마다 기저 벡터 $\varphi_j = (u^2, uv, v^2, u, v, 1)$ 을 만들면 이 문제는 6×6 연립방정식 (정규방정식)으로 정리됩니다:

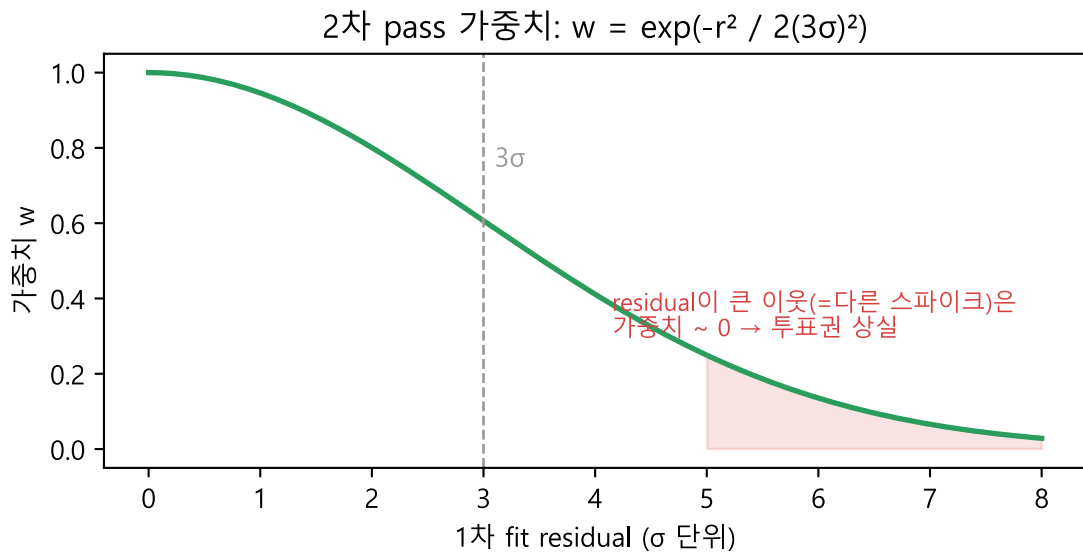
$$(\sum_j \varphi_j \varphi_j^T) \cdot x = \sum_j \varphi_j w_j, \quad x = (a, b, c, d, e, f)^T$$

6×6 이라 Eigen의 LDLT 분해로 마이크로초 안에 풀립니다. 점 하나당 비용이 아주 작아서 수십만 점도 몇 초면 끝납니다(전 점 병렬). 행렬이 퇴화하면(이웃이 한 직선 위에 있는 등) 평면 residual로 안전하게 후퇴합니다.

5 Robust 2-pass — 스파이크끼리 서로 감싸주지 못하게

이웃 K개 안에 다른 스파이크가 섞여 있으면 어떻게 될까요? 최소제곱은 오차의 "제공"을 줄이므로 큰 오차를 내는 점(스파이크)에 과민하게 반응해 곡면이 그쪽으로 달려갑니다. 그래서 두 번 fit합니다:

- **1차 pass:** 모든 이웃을 동등하게 fit → 각 이웃의 residual r_j 측정
- **2차 pass:** r_j 가 큰 이웃의 가중치를 $w = \exp(-r^2/2(3\sigma)^2)$ 로 깎아서 다시 fit. σ 는 이웃 residual들의 중앙값 기반(MAD) 척도라, 절반 이상이 오염되지 않는 한 믿을 수 있습니다.



결과: 창 안의 다른 스파이크는 기준면에 대한 **투표권을 잃고**, 기준면은 진짜 표면에 남습니다. (완전히 매끈한 데이터에서는 σ 가 0에 가까워져 모든 가중치가 죽는 사고가 날 수 있어, σ 에 이웃 퍼짐 폭의 10^{-4} 배라는 하한을 둡니다.)

6 Residual — "이 점은 얼마나 튀어나왔나"

완성된 기준 곡면에 검사 대상 점을 대조합니다:

$$r_i = w_i - q(u_i, v_i)$$

r_i 는 **부호 있는** 값입니다 — 양수면 법선 위쪽으로 돌출, 음수면 표면 안쪽으로 함몰. 정상 점이라면 측정 노이즈 수준(거의 0), 스파이크라면 돌출 높이 그대로가 나옵니다.

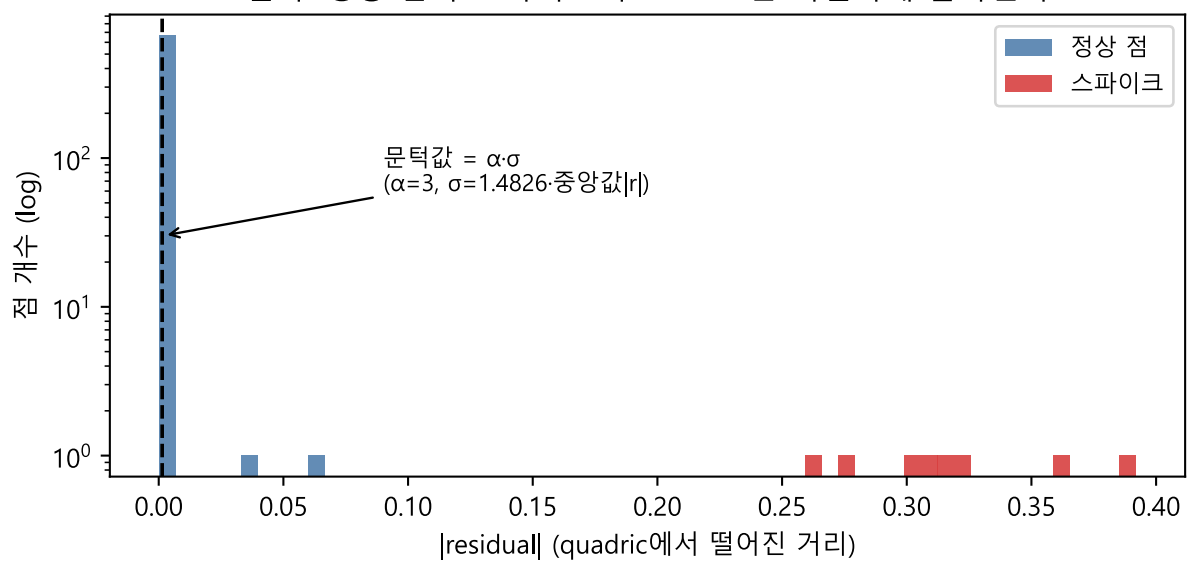
7 전역 문턱값 — 통계로 "튀어나옴"을 정의

이제 전체 구름의 $|r_i|$ 분포에서 선을 긋습니다. 평균·표준편차를 쓰면 안 됩니다 — **outlier** 자신이 평균과 표준편차를 끌어올려 문턱이 물러집니다(우리가 잡으려는 범인이 배심원에 앉는 꼴). 대신 순위 기반의 강건한 척도를 씁니다:

$$\sigma = 1.4826 \times \text{median}(|r_i|) \rightarrow |r_i| > \alpha \cdot \sigma \text{ 이면 outlier}$$

중앙값은 절반이 오염돼도 흔들리지 않고, 1.4826은 정규분포에서 MAD를 표준편차 단위로 환산하는 상수입니다. 즉 α (기본 3)는 "이 스캔의 전형적 노이즈의 몇 배나 튀어나와야 제거하는가"라는, 단위와 무관한 직관적인 손잡이가 됩니다.

실측: 정상 점과 스파이크의 residual은 확실하게 갈라진다



심화: 이차곡면 최소제곱 fit — 밑바닥부터 코드까지

4단계의 "이차곡면 최소제곱 fit"이 QFOR의 심장입니다. 이 장은 그 한 단계를 아무 배경지식 없이 따라올 수 있게 처음부터 다시 만듭니다.

심화 1. 우리가 찾는 것은 숫자 6개다

이웃 점들이 로컬 좌표 (u_j, v_j, w_j) 로 준비되어 있습니다 (u, v = 접평면 위 위치, w = 표면에서 켄 높이). 우리는 이 점들 위로 지나가는 부드러운 2차 곡면

$$q(u, v) = a \cdot u^2 + b \cdot uv + c \cdot v^2 + d \cdot u + e \cdot v + f$$

를 긋고 싶습니다. 곡면의 모양은 계수 **(a, b, c, d, e, f)** — **숫자 6개**로 완전히 결정됩니다. 즉 "곡면을 fit한다"는 말은 **숫자 6개를 고르는 문제**입니다. 고르는 기준이 최소제곱입니다.

심화 2. 워밍업 — 왜 "오차 제곱의 합"인가

이웃 j 에 대해 곡면이 예측하는 높이는 $q(u_j, v_j)$, 실제 높이는 w_j . 그 차이가 잔차(residual)입니다:

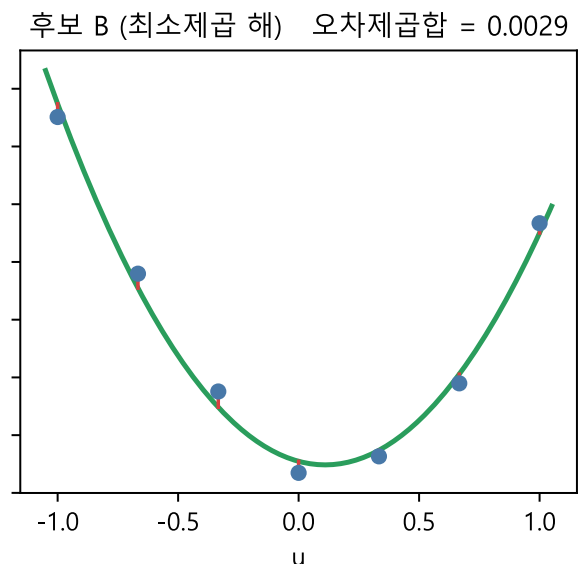
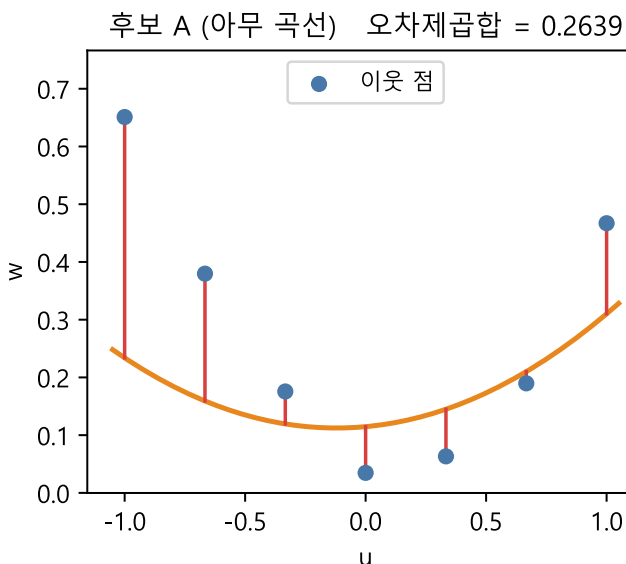
$$r_j = w_j - q(u_j, v_j)$$

"모든 이웃에 잘 맞는 곡면"을 수식 하나로 만들려면 잔차들을 하나의 점수로 합쳐야 합니다. 그냥 더하면 +잔차와 -잔차가 상쇄되어 엉터리 곡면이 만점을 받을 수 있으니, **제곱해서** 더합니다:

$$SSE(a..f) = \sum_j r_j^2 \quad (\text{오차제곱합, Sum of Squared Errors})$$

제곱을 쓰는 이유는 세 가지 — ① 부호 상쇄 방지 ② 어디서나 미분 가능(최솟값을 미분=0으로 찾음) ③ 큰 오차에 큰 벌점(곡면이 어느 한 점만 크게 놓치는 걸 싫어하게 됨). SSE가 가장 작아지는 6개 숫자가 "최소 제곱 해"입니다.

빨간 세로선(residual)의 제곱을 전부 더한 값이 가장 작은 곡선을 고른다



심화 3. 미분 = 0 → 연립방정식 (정규방정식)

SSE는 a..f 각각에 대한 **2차식**이라 그릇(볼록) 모양 — 최솟값이 딱 하나이고, 그 지점에서 6개 편미분이 모두 0입니다. 예로 a에 대한 편미분을 손으로 해 보면:

$$\partial \text{SSE} / \partial a = \sum_j 2 \cdot r_j \cdot (-u_j^2) = 0 \rightarrow \sum_j u_j^2 \cdot q(u_j, v_j) = \sum_j u_j^2 \cdot w_j$$

같은 일을 b, c, d, e, f에도 하면 **미지수 6개, 식 6개**의 연립방정식이 나옵니다. 이걸 깔끔하게 쓰는 도구가 **기저 벡터** ϕ 입니다. 각 이웃마다

$$\phi_j = (u_j^2, u_j v_j, v_j^2, u_j, v_j, 1)^T \text{ 이면 } q(u_j, v_j) = \phi_j \cdot x, \quad x = (a, b, c, d, e, f)^T$$

이 표기로 연립방정식 6개가 한 줄이 됩니다 — 이것이 **정규방정식(normal equations)**:

$$(\sum_j \phi_j \phi_j^T) \cdot x = \sum_j \phi_j \cdot w_j \quad \text{즉} \quad A \cdot x = b, \quad A \text{는 } 6 \times 6$$

$\phi \phi^T$ 는 6×6 행렬(모든 기저쌍의 곱), $\phi \cdot w$ 는 길이 6 벡터. 이웃마다 이 둘을 만들어 **누적해서 더하기만** 하면 A와 b가 완성됩니다.

심화 4. 손으로 푸는 미니 예제 (1차원 버전)

감을 잡기 위해 미지수를 3개로 줄인 1차원 파라볼라 $w = a \cdot u^2 + d \cdot u + f$ 를 5개 점으로 fit해 봅니다. 점: $u = -1, -0.5, 0, 0.5, 1$, 그리고 $w = u^2$ (정확히 파라볼라 위). 기저는 $\phi = (u^2, u, 1)$.

필요한 합	$\sum u^4$	$\sum u^3$	$\sum u^2$	$\sum u$	N	$\sum u^2 w$	$\sum u w$	$\sum w$
값	2.125	0	2.5	0	5	2.125	0	2.5

$$A \cdot x = b: \begin{bmatrix} 2.125 & 0 & 2.5 \\ 0 & 2.5 & 0 \\ 2.5 & 0 & 5 \end{bmatrix} \begin{bmatrix} a \\ d \\ f \end{bmatrix} = \begin{bmatrix} 2.125 \\ 0 \\ 2.5 \end{bmatrix}$$

2행에서 $d = 0$. 1행과 3행을 연립하면 (3행÷2: $1.25a + 2.5f = 1.25 \rightarrow f = 0.5(1-a)$; 1행에 대입: $2.125a + 1.25(1-a) = 2.125 \rightarrow 0.875a = 0.875$) → **a = 1, d = 0, f = 0**. 데이터가 정확히 $w = u^2$ 위에 있었으니 fit이 그걸 그대로 복원한 것 — 검산 완료. QFOR의 6×6 은 이것과 완전히 같은 구조가 2차원으로 커진 것뿐입니다.

심화 5. 실제 코드 — 6×6 누적과 LDLT

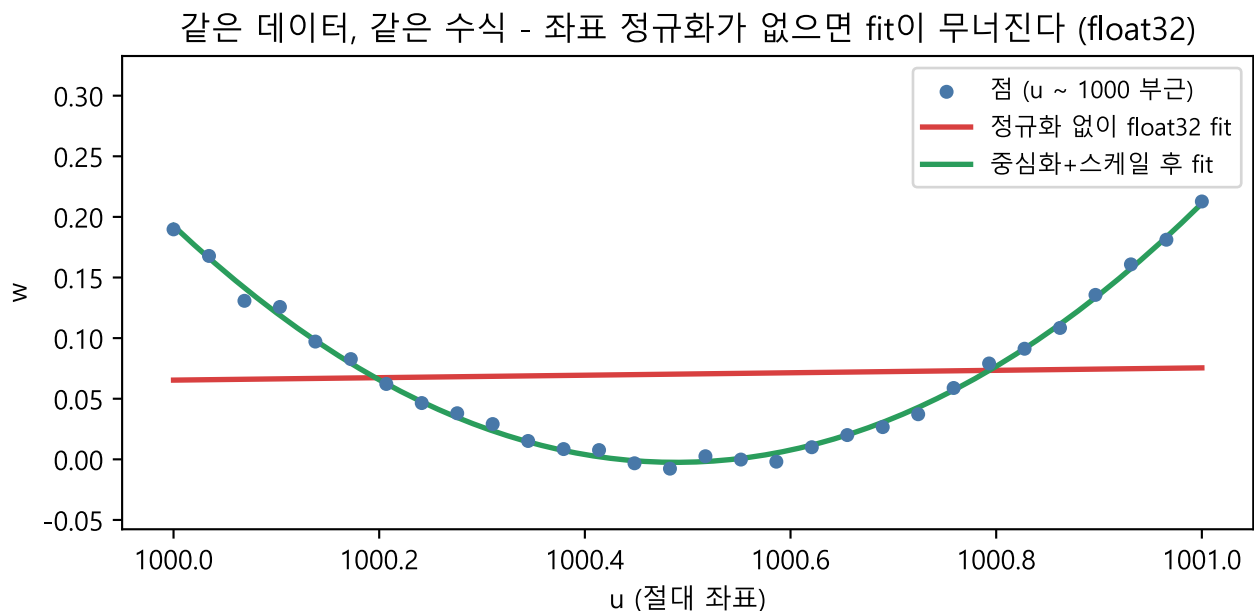
Orange의 실제 구현(`modes.cpp`의 `qforResiduals`)에서 fit 부분만 추려 주석을 달았습니다. $N \times 6$ 행렬을 만들어 두는 대신 이웃 하나 처리할 때마다 6×6 에 **바로 누적**합니다 — 이웃이 몇 개든 메모리는 6×6 하나로 끝나고, 점 하나당 비용이 일정해서 수십만 점 병렬 처리에 딱 맞습니다.

```
// 이웃들의 로컬 좌표 loc[t] = (u, v, w)와 검사점 (ui, vi, wi)는 준비됨.
// rob[t] = robust 가중치(1차 pass 후 스파이크 이웃은 0에 가까움).
Eigen::Matrix<float, 6, 6> A = Eigen::Matrix<float, 6, 6>::Zero();
Eigen::Matrix<float, 6, 1> b = Eigen::Matrix<float, 6, 1>::Zero();
for (size_t t = 0; t < oth.size(); ++t) {
    float u = loc[t].x(), v = loc[t].y();
    Eigen::Matrix<float, 6, 1> phi;
    phi << u * u, u * v, v * v, u, v, 1.0f; // 기저  $\phi$ 
    A += rob[t] * (phi * phi.transpose()); // 6x6 누적:  $\sum w \cdot \phi \phi^T$ 
    b += rob[t] * (phi * loc[t].z()); // 길이6 누적:  $\sum w \cdot \phi \cdot w_j$ 
}
Eigen::LDLT<Eigen::Matrix<float, 6, 6>> ldlt(A); //  $A \cdot x = b$  풀기
if (ldlt.info() == Eigen::Success) {
    Eigen::Matrix<float, 6, 1> q = ldlt.solve(b); //  $q = (a, b, c, d, e, f)$ 
    // 검사점의 residual: 실제 높이 - 곡면이 예측한 높이
    d = wi - (q[0]*ui*ui + q[1]*ui*vi + q[2]*vi*vi + q[3]*ui + q[4]*vi + q[5]);
}
```

왜 LDLT인가? $A = \sum \phi \phi^T$ 는 항상 대칭이고 음이 아닌 고유값만 갖는(양의 준정부호) 특별한 행렬입니다. 이런 행렬 전용 분해가 LDLT — 일반 가우스 소거보다 빠르고(6×6이면 수십 나노초), 피벗 교환 없이도 수치적으로 안정합니다. 이웃들이 한 직선 위에 놓이는 등 정보가 부족하면 `ldlt.info()`가 실패를 알려주고, 그때는 평면 residual로 안전하게 후퇴합니다.

심화 6. 좌표 정규화 — 조용히 전부 망가뜨리는 함정

코드에서 u, v 를 이웃 퍼짐 폭으로 나누는($\ast s$) 한 줄이 있습니다. 사소해 보이지만 빼면 이렇게 됩니다:



이유: A에는 u^4 크기의 항(ϕ 의 u^2 끼리의 곱)과 1 크기의 항이 같은 행렬 안에 섞여 있습니다. 점의 좌표가 원점에서 1000쯤 떨어져 있으면 $u^4 \sim 10^{12}$ 이고, float의 유효자릿수(~7자리)로는 큰 항에 작은 항이 통째로 잡아먹혀 A가 사실상 특이행렬이 됩니다(조건수 폭발). 위 그림의 빨간 곡선이 그 결과 — 같은 데이터, 같은 수식인데 fit이 직선으로 무너졌습니다. **중심을 빼고(centroid) 퍼짐 폭으로 나누면** u, v 가 항상 $\pm 1 \sim 2$ 범위

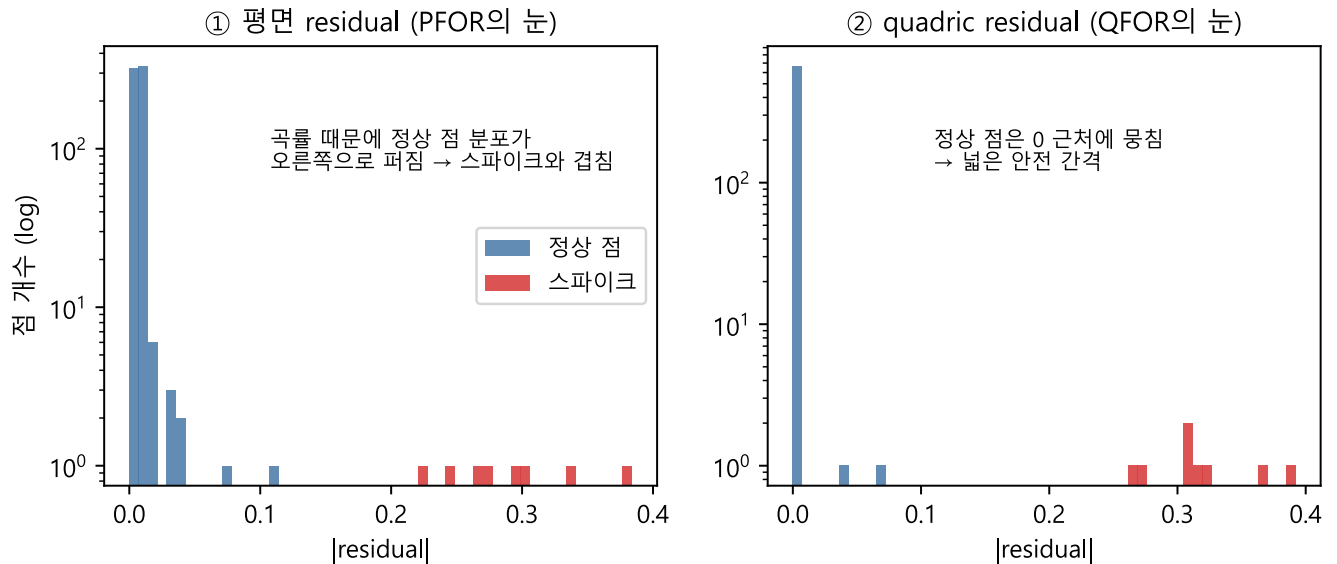
라 u^4 도 $O(1)$ — 어떤 단위의 모델이 와도(mm든 km든) 행렬이 건강합니다. QFOR가 PCA 중심 c 를 원점 삼아 로컬 좌표로 변환하고 스프레드로 나누는 것이 정확히 이 처치입니다.

가중치 한 줄로 **robust**까지

코드의 `rob[t]`가 유일한 추가 장치입니다: SSE를 $\sum w_t \cdot r_t^2$ 로 바꾸면 유도 전체가 그대로 성립하고, A 와 b 의 누적에 가중치만 곱으면 됩니다. 1차 fit에서 크게 벗어난 이웃(다른 스파이크)에 $w \approx 0$ 을 주고 다시 풀면 — 그것이 4단계의 robust 2-pass입니다. 최소제곱의 뼈대는 그대로 두고 "누가 투표하는가"만 바꾸는 것.

5. 실측 비교: PFOR의 눈 vs QFOR의 눈

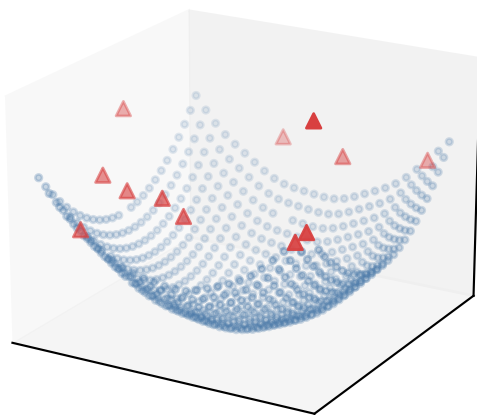
같은 데이터(곡면 + 잔물결 노이즈 + 스파이크 8개, 676점)를 두 기준면으로 본 residual 분포입니다. 이 문서의 그래프는 실제 QFOR 구현과 동일한 로직의 시뮬레이션으로 계산했습니다.



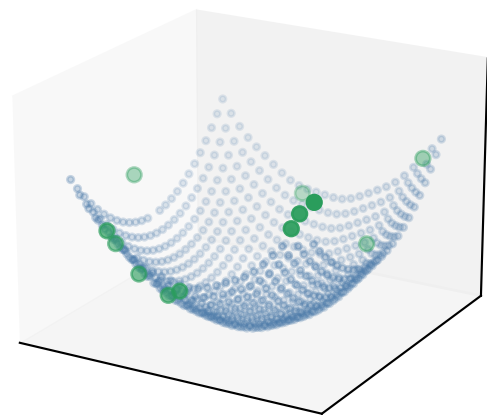
평면 기준(왼쪽)에서는 곡률 때문에 정상 점의 분포가 오른쪽으로 넓게 퍼져 스파이크와 겹칩니다 — 어디에 선을 그어도 오검출 또는 미검출이 생깁니다. 이차곡면 기준(오른쪽)은 정상 점을 0 근처로 압축해 두 무리 사이에 넓은 간격을 만듭니다. 이 간격이 QFOR의 존재 이유입니다.

위 시뮬레이션의 판정 성적: 스파이크 8/8 검출, 오검출 4/668 (0.6%). (Orange의 단위 테스트 `test_qfor`도 동일 구조의 데이터로 전 스파이크 검출 + 오검출 <5%를 회귀 검증합니다.)

검출 결과 (빨강 = outlier 판정)



Fit 적용 후 (outlier를 표면 위로 투영)



6. "Fit" — 지우는 대신 되돌려 놓기

outlier를 삭제하면 그 자리에 구멍이 남습니다. QFOR의 **Fit** 동작은 대신 outlier를 자기 기준 곡면 위로 **법선 방향으로 residual만큼 되밀어** 놓습니다:

$$p_i' = p_i - n \cdot r_i$$

정상 점은 그대로 두고 판정된 outlier만 이동하므로, 점 개수를 보존하면서 표면을 복원합니다. Orange에서는 실행 취소(undo)가 가능합니다.

7. 파라미터 가이드

파라미터	기본값	의미와 조절 요령
K Neighbors	24 (10–256)	기준면을 만들 증인 수. 크게 — 기준면이 안정되고 노이즈에 둔감해지지만, 창이 넓어져 미세한 형상 디테일까지 "곡률"로 흡수 못 하고 돌출로 오인할 수 있음. 작게 — 디테일 보존은 좋지만 스파이크 몇 개에 기준면이 흔들림. 미지수 6개 대비 최소 4배(24개)가 무난한 출발점.
Alpha	3.0 (1–10)	"전형적 노이즈(σ)의 몇 배부터 outlier인가". 3 = 통계의 관례적 3σ 규칙. 덜 잡히면 2.0~2.5로 내리고, 정상 점까지 잡히면 4~5로 올리세요. 데이터의 절대 크기와 무관하게 동작하므로 모델 단위를 신경 쓸 필요가 없습니다.

8. Orange에서 사용하기

- 포인트 클라우드 선택 → 메뉴 **Geometry** ▶ **(Filter 그룹)** ▶ **Outlier: QFOR**. 초록 = 유지, 빨강 = outlier 판정으로 원본 점 색이 즉시 갈아입혀집니다(모드 해제 시 원복).
- 파라미터 다이얼로그에서 **K Neighbors / Alpha** 슬라이더 조절 — 결과가 실시간 재계산됩니다(백그라운드 스레드).
- 다이얼로그의 **Fit** 버튼 = 6장의 투영 복원(undo 가능).
- **Pipeline Design** 노드 캔버스에서 points→points 스테이지로 체인 가능 (예: QFOR → Smooth → Reconstruct).
- 단축키 **M**으로 모드 순환.

9. 한계와 실전 팁

이런 경우는 주의

- **날카로운 모서리(crease)** — 2차식은 접힌 면을 표현하지 못해 모서리 능선을 돌출로 오인할 수 있습니다. 모서리가 중요한 CAD류 데이터라면 α 를 올리거나 K를 줄이세요.
- **극단적 밀도 불균일** — kNN 창 의 물리적 크기가 지역마다 달라져 σ 의 의미가 흔들립니다. 이때는 SOR로 고립점을 먼저 정리한 뒤 QFOR를 거는 2단 파이프라인이 효과적입니다.
- **돌출 덩어리(blob)** — 점 하나가 아니라 수십 점짜리 혹이 붙어 있으면, 혹 내부 점들의 이웃은 혹 자신이라 기준면도 혹을 따라갑니다. 이걸 Bump: Detect/Remove(트림 평면 + 성장) 담당입니다.

방법 선택 치트시트

- 허공에 뜬 고립점, 스캐너 미스샷 → **SOR / ROR**
- 평면 위주 모델의 표면 돌출 → **PFOR** (가볍고 충분)
- 곡면 모델의 표면 돌출 (치아, 유기 형상, 스캔 일반) → **QFOR**
- 표면에 붙은 돌출 "덩어리" → **Bump: Detect / Remove**
- 돌출 + 거친 질감 구분이 필요한 까다로운 경우 → **Protrusion (MLS+PFOR)**

10. 요약

질문	QFOR의 답
기준 표면은?	이웃 K개로 fit한 이차곡면 (곡률을 흡수)
내가 기준을 오염시키면?	Leave-one-out — 검사 대상은 fit에서 제외
이웃에 다른 outlier가 있으면?	Robust 2-pass — 큰 residual 이웃의 가중치 삭감
얼마나 튀어야 outlier?	전역 중앙값/MAD 척도의 α 배 (기본 3σ)
지우면 구멍이 남는데?	Fit — 법선 방향으로 곡면 위에 재투영 (undo 가능)
구현 위치	<code>engine/core/src/modes/modes.cpp</code> — <code>qforResiduals</code> / <code>qforColors</code> / <code>qforFit</code> , 테스트 <code>test_qfor</code>